KIAN'S DEV SKILL TRACK · PRACTICAL MASTERY GUIDE · 2026

GITHUB

Version Control & Collaboration — Complete Mastery Reference

Scope: Git + GitHub

Level: Beginner → Professional

Environment: Linux (Debian/XFCE4) · VS Code · gh CLI

Type: Practical Skill Guide

CHAPTER 00

Overview & Mastery Priority Map

This guide covers Git (the version control tool) and GitHub (the hosting platform built on top of it) end to end — every command, button, and workflow you need to work confidently solo, with classmates on group projects, and eventually on professional

teams. It is not tied to a single QCU subject, but it directly supports every course that involves a repo: WS101/WS102 web projects, CC103 onward, and especially IPT101/IPT102, SIA101/102, and CAP101/CAP102 where group repos and version history are graded artifacts in their own right.

Why This Matters Beyond One Subject

Every BSIT group project from here on assumes you can push code without breaking a teammate's work, open a pull request without panic, and recover from a mistake without losing history. This is infrastructure knowledge — once mastered, it stops being something you think about and becomes muscle memory, the same way `git status` already is on your XoDos setup.

Mastery Priority Map

Topic	Chapter	Frequency of Use	Difficulty
Daily workflow (status/add/commit/push)	03	Constant	Easy

GITHUB · DEV SKILL TRACK

Version Control & Collaboration Mastery

Complete Reference · 2026

INTRODUCTION

CHAPTERS

Topic	Chapter	Frequency of Use	Difficulty
Branching & merging	04	Constant	Medium
Remotes & syncing	05	Constant	Easy
Pull requests & forking	06	Every group project	Medium
Web UI features (Issues, Actions, Pages)	07	Weekly	Easy
Undoing mistakes	08	Occasional, high-stakes	Hard
GitHub CLI (gh)	09	Optional convenience	Easy
Team collaboration norms	10	Every group project	Medium
Professional standards & security	11	Before job hunting	Easy

GITHUB · DEV SKILL TRACK

Version Control & Collaboration Mastery

Complete Reference · 2026

INTRODUCTION

CHAPTERS

How To Use This Guide

Chapters 01–05 are the non-negotiable core — read those first and drill them until they're automatic. Chapters 06–07 you'll need the first time you do a real group project. Chapter 08 you'll need exactly when you're panicking, so it's worth skimming once now so you know it exists. Chapters 09–11 are what separates "can use Git" from "works like a professional."

CHAPTER 01

Git vs GitHub & Setup

The single most important distinction to internalize before anything else: Git and GitHub are not the same thing, and confusing them causes real workflow mistakes down the line.

LOCAL TOOL

Git

A version control program that

CLOUD PLATFORM

GitHub

A website that hosts Git repositories

runs entirely on your machine. It tracks every change to your files over time, lets you branch off to try things safely, and works completely offline. Git existed before GitHub and works the same way whether or not you ever touch GitHub.

online and adds collaboration features on top: pull requests, code review, issue tracking, project boards, CI/CD via Actions, and a public profile. GitLab and Bitbucket are competitors that do the same job.

The Three States (Mental Model)

Every file in a Git-tracked project sits in one of three places at any moment. Understanding this is what makes every other command make sense instead of feeling like memorized incantations.

1 **Working Directory**

Your actual files on disk, exactly as you're editing them in VS Code right now. Nothing here is tracked by history yet.

2 **Staging Area (the "Index")**

A holding area for changes you've explicitly marked as ready with `git add`. This is Git's way of letting you build a commit out of only some of your changes, not necessarily everything you've touched.

3 **Repository (committed history)**

Once you run `git commit`, the staged snapshot becomes a permanent, named point in history that you can always return to.

The Essay Analogy

Working directory is your draft as you're typing it. Staging is highlighting the paragraphs you've decided are final for this revision. Committing is saving that revision with a version label. GitHub, in this analogy, is Google Drive — a place you upload the saved revisions so others can read them or work on their own copy.

First-Time Setup (Per Machine)

You've already done this in your Linux dev environment — this is the reference for when you set up Git on any new machine.

BASH

```
git config --global user.name "Y
git config --global user.email "
git config --global init.defaul
git config --global core.editor
git config --list
```

Authentication: HTTPS vs SSH vs PAT

GitHub removed plain password authentication for Git operations in 2021 — you must use either a Personal Access Token (PAT) over HTTPS or an SSH key. Your XoDos setup already uses PAT auth via gh.

Method	How it works	Best for
PAT over HTTPS	Token pasted in place of a password on first push, or cached via `gh auth login`	Simplest setup, works everywhere, easy to revoke/rotate
SSH key	Key pair generated once, public key added to GitHub account, no token	Frequent pushing from a stable machine, no repeated prompts

Method	How it works	Best for
	needed per-push	
gh CLI auth	`gh auth login` handles token creation and storage for you	Terminal-first workflow — this is what you're using now

Wrong scope	Creating a PAT without the `repo` scope checked, then wondering why pushes fail with a permissions error.
Global vs local	Forgetting that `git config` without `--global` only applies to the current repo — useful when a specific project needs a different email (e.g. a work vs school account), but confusing if done by accident.
Skipped setup	Cloning a repo on a fresh machine and committing before running `git config` — commits get attributed to a generic or wrong identity.

CHAPTER 02

Repositories & Local Structure

A repository ("repo") is a project folder that Git is tracking. You either start one from scratch or copy one that already exists on GitHub.

Starting a Repository

1. **Option A — Start fresh locally**
`mkdir my-project && cd my-project` then `git init`. This creates a hidden `.git` folder that holds the entire history — never edit its contents by hand.

2. **Option B — Clone an existing repo**
`git clone https://github.com/username/repo-name.git` downloads the full project plus its entire commit history in one step.

Typical Project Structure

```
my-project/
├── .git/              ← hidden, Git's internal
├── .gitignore         ← tells Git what NOT to t
├── README.md          ← what the project is and
├── LICENSE             ← optional: usage terms
```

```
├── src/                ← your actual source cod
└── .vscode/            ← editor settings (usual
```

.gitignore — Keeping Noise Out of History

Without a `.gitignore`, Git will happily track build output, IDE config, and — worst case — secrets. Create this file at the project root before your first commit.

.GITIGNORE — JAVA/WEB PROJECT EXAMPLE

```
*.class
target/
.vscode/
node_modules/
.env
.DS_Store
*.log
```

DOCUMENTATION

README.md

The first thing anyone sees — classmate, professor, or future employer. Should explain what the project does, how to run it, and any setup steps. GitHub renders this automatically

LEGAL

LICENSE

Defines how others may use your code. Optional for school assignments, but standard practice for anything public — MIT is the common permissive

on the repo's front page.

default. Covered in more depth in Chapter 11.

Never Commit Secrets

API keys, passwords, database credentials, and `.env` files should never enter Git history — even in a private repo. Add them to `.gitignore` before your first commit, not after. Once something is committed, it exists in history even if you delete it in a later commit (see Chapter 08 for what to do if this already happened).

CHAPTER 03

Daily Workflow & Commit Craft

This is the loop you'll run dozens of times a day once you're actively developing. Master this chapter cold — everything else builds on it.

GITHUB · DEV SKILL TRACK

Version Control & Collaboration Mastery

Complete Reference · 2026

The Core Loop

BASH

```
git status
git add filename.java
git add .
git commit -m "Add login validat
git push
```

Habit To Build Now

Run `git status` before and after almost every other Git command. It tells you exactly what's staged, unstaged, or untracked at any point — there's no guessing, and it catches mistakes (like about to commit a file you didn't mean to) before they happen.

Staging In More Detail

Command	What it does
`git add file.java`	Stages one specific file
`git add .`	Stages everything changed in the current directory and below
`git add -p`	Interactive patch mode — stage individual chunks within a file, not

Command	What it does
	the whole file at once
`git diff`	Shows exact line changes that are NOT yet staged
`git diff --staged`	Shows exact line changes that ARE staged, about to be committed

Viewing History

BASH

```
git log
git log --oneline
git log --oneline --graph --all
```

Writing Commit Messages That Don't Embarrass You Later

A commit message is documentation for your future self and your teammates. The **Conventional Commits** format is widely used professionally and scales well to group projects:

FORMAT

type(scope): short description

Conventional Commits specification

Common types: `feat` (new feature), `fix` (bug fix), `docs` (documentation only), `refactor` (no behavior change), `style` (formatting only), `chore` (maintenance/tooling). Example: `fix(auth): resolve null pointer in Student.java constructor`

Vague	`"fix stuff"`, `"asdf"`, `"update"` — tells nobody, including future you, what actually changed.
Version-in-message	`"final version 2 REAL FINAL"` — Git already tracks versions; this is a sign you need branches (Chapter 04), not filename-style commit messages.
Giant commits	One commit containing an entire week of unrelated changes. Prefer small, frequent, logically-grouped commits — easier to review, easier to revert individually if needed.

CHAPTER 04

Branching & Merging

A branch is an independent line of work. You can experiment or build a feature without touching working code on `main` — merge it back in once it's ready, or discard it if it doesn't pan out.

Working With Branches

BASH

```
git branch
git branch feature-login
git checkout feature-login
git checkout -b feature-login
git switch feature-login
git switch -c feature-login
```

Merging

BASH

```
git checkout main
git merge feature-login
```

SIMPLE CASE

COMMON CASE

Three-Way Merge

Fast-Forward Merge

When `main` hasn't changed since you branched off, Git just moves the `main` pointer forward to include your commits. No new merge commit is created — the history stays linear.

When `main` has moved forward with other commits since you branched, Git creates a new **merge commit** that combines both histories. This is what you'll see most often on active group projects.

Resolving Merge Conflicts

A conflict happens when both branches changed the same lines. Git marks the conflicting section directly in the file — it will not guess for you.

CONFLICT MARKERS

```
<<<<<<< HEAD
your version of the code
=======
their version of the code
>>>>>>> branch-name
```

1. **Open the flagged file**
   VS Code highlights conflict markers automatically and offers "Accept Current / Accept Incoming / Accept Both" buttons inline.

2. **Decide the correct final version**
   Keep one side, the other, or manually combine both — then delete the `<<<` / `===` / `>>>` marker lines entirely.

3. **Stage and finish the merge**
   Run `git add filename` then `git commit` — Git already pre-fills a merge commit message for you.

Rebase (Advanced, Optional)

The Golden Rule of Rebase

`git rebase` replays your commits on top of another branch instead of creating a merge commit, producing cleaner linear history. It's powerful but rewrites commit history — never rebase a branch that has already been pushed and that others may have pulled from. For QCU group work, plain `merge` is the safer default; save rebase for solo feature branches you haven't shared yet.

Deleting Branches

BASH

```
git branch -d feature-login
git branch -D feature-login
```

Wrong branch	Committing directly on `main` when you meant to be on a feature branch — check `git status` or your VS Code branch indicator before committing.
Stale branches	Never deleting merged branches — clutters the branch list and makes it harder to tell what's actually active.
Not pulling first	Merging without pulling `main`'s latest changes first, causing avoidable conflicts that a quick `git pull` beforehand would have surfaced earlier and more cleanly.

CHAPTER 05

Remotes & Syncing

A remote is a saved reference to a repository's URL on GitHub. When you clone a repo, Git automatically names that remote `origin`.

Managing Remotes

BASH

```
git remote -v
git remote add origin https://gi
git push -u origin main
```

What "-u" Actually Does

The `-u` flag (short for `--set-upstream`) links your local `main` branch to `origin/main` permanently. After this one-time setup, plain `git push` and `git pull` know exactly where to send and fetch from without needing the branch name spelled out every time.

Fetch vs Pull

Command	What it does
`git fetch`	Downloads new commits from GitHub into your local copy of the remote branch, but does NOT touch your working files or current branch

Command	What it does
`git pull`	`fetch` + automatically merges the downloaded commits into your current branch

`git fetch` first is the safer habit when working with others — it lets you review exactly what changed (`git log origin/main`) before deciding to merge it into your own work.

Multiple Remotes (Upstream, for Forks)

When you fork someone else's repo, your fork gets its own `origin` — but you also add their original repo as a second remote, conventionally named `upstream`, so you can pull in their updates without losing your own commits.

BASH

```
git remote add upstream https://
git remote -v
# origin    → your fork (read/wr
# upstream  → the original repo
```

Pull Requests & Forking

A pull request (PR) is a request to merge one branch into another — the standard way to propose, review, and discuss changes before they become official. This is the single most important collaboration mechanic on GitHub.

The Pull Request Lifecycle

1. **Push your branch**
   `git push -u origin your-branch-name`

2. **Open the PR**
   On GitHub, click "Compare & pull request" — appears automatically after a branch push.

3. **Write a clear title and description**
   What changed, why, and anything reviewers should pay special attention to. A good PR description saves your reviewers time and shows professionalism.

4 **Review round**

Teammates comment, request specific changes, or approve. Push additional commits to the same branch to address feedback — they appear on the same PR automatically.

5 **Merge**

Once approved, click "Merge pull request." Choose a merge strategy (below), then optionally delete the now-merged branch.

Draft Pull Requests

Open a PR as a **draft** when you want early visibility or feedback on work-in-progress without signaling it's ready to merge. Convert it to a regular PR once it's actually complete.

Merge Strategies

DEFAULT

Merge Commit

Preserves full branch history plus an explicit merge commit. Most transparent option — you can see exactly when and how each branch came in.

CLEAN HISTORY

Squash and Merge

Combines all commits from the branch into a single commit on `main`. Great for feature branches with messy "wip", "fix typo" commits you don't need

	preserved individually.

LINEAR

Rebase and Merge

Replays each commit individually onto `main` with no merge commit at all — keeps history perfectly linear, but loses the visual grouping of "these commits belonged to one feature."

For QCU group projects, **squash and merge** is usually the easiest default — one clean commit per feature/PR on `main`, regardless of how messy the branch's internal history was.

Forking vs Branching

	Branch	Fork
Where it lives	Same repository	Your own full copy of someone

	Branch	Fork
		else's repository
Requires	Write access / being added as a collaborator	Nothing — anyone can fork a public repo
Use case	Team members with write access to a shared repo	Contributing to a repo you don't own (open source)
Merging back	PR from branch to `main`, same repo	PR from your fork to the original repo (`upstream`)

Which One for QCU Group Projects

When everyone is added as a collaborator on one shared repo (the normal setup for a group project), branching is simpler than forking — skip forking entirely. Save forking for contributing to a public repo you don't have write access to.

Contributing to an Open-Source Repo (Fork Workflow)

1. **Fork on GitHub**
   Click "Fork" on the original repo's page — creates a full copy under your account.

2. **Clone your fork**
   `git clone https://github.com/YOUR-username/repo.git`

3. **Add the original as upstream**
   `git remote add upstream https://github.com/original-owner/repo.git`

4. **Branch, commit, push to YOUR fork**
   `git push origin your-branch-name` — never push directly to `upstream`, you don't have write access anyway.

5. **Open a PR from your fork to upstream**
   GitHub detects the fork relationship and offers this automatically on your fork's page.

Keeping Your Fork in Sync

BASH

```
git fetch upstream
git checkout main
```

```
git merge upstream/main
git push origin main
```

CHAPTER 07

GitHub Web UI Features

Beyond the code itself, GitHub bundles a full set of project-management and automation tools directly into every repository. Most are underused by students but expected in professional settings.

TRACKING

Issues

Track bugs, tasks, or ideas with labels, assignees, and milestones. Even solo, this keeps a project's to-do list visible and organized instead of scattered across notes apps.

PLANNING

Projects (Boards)

Kanban-style boards (To Do / In Progress / Done) that link directly to Issues and PRs — useful for visualizing a group project's status at a glance during a meeting.

GitHub Actions (CI/CD)

Automate tasks that run on GitHub's servers every time you push — compiling code, running tests, or deploying automatically. Configured via a YAML file in `.github/workflows/`.

.GITHUB/WORKFLOWS/BUILD.YML

```
name: Build and Test
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v
      - name: Set up JDK 21
        uses: actions/setup-java
        with:
          java-version: '21'
          distribution: 'temurin
      - run: mvn test
```

More advanced — worth exploring once a project outgrows single-file assignments, but the mental model matters even before you write your first workflow: every push can automatically trigger a build/test cycle.

GitHub Pages

Free static site hosting straight from a repo — perfect for WS101/WS102 HTML/CSS projects. Settings → Pages → choose branch → site goes live at `username.github.io/repo-name`.

Username Change Caveat

Renaming your GitHub username later will NOT redirect existing Pages sites to the new URL — worth keeping in mind before deploying anything you plan to link long-term (e.g. on a resume or portfolio).

Releases & Tags

A tag marks a specific commit as a named version point (e.g. `v1.0.0`); a Release wraps a tag with release notes and optional downloadable files.

BASH

```
git tag v1.0.0
git push origin v1.0.0
```

Gists
Quick sharing

Discussions
Q&A / forum

Wiki
Long-form docs

Share a single file or snippet without creating a whole repo — good for a code snippet in a forum post or to a classmate.

Threaded conversation space separate from Issues — for questions and ideas that aren't bugs or tasks.

Built-in wiki pages per repo — useful for documentation too long or structured for a single README.

Notifications: Watch, Star, Subscribe

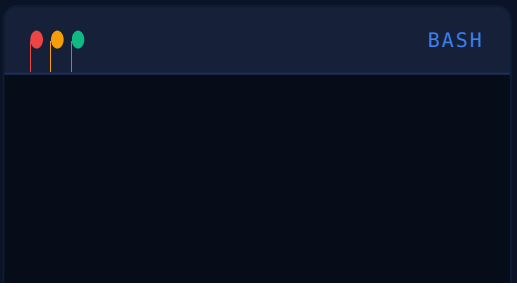
Action	What it does
Star	Bookmark a repo to your profile — purely a personal reference, no notifications
Watch	Get notified of all activity (issues, PRs, discussions) on a repo
Subscribe	Get notified on one specific Issue or PR thread without watching the whole repo

Undoing Mistakes

Every developer breaks things. What separates confidence from panic is knowing exactly which undo command applies to which situation — the right tool depends entirely on how far the mistake has traveled.

Situation	Command
Uncommitted changes, want to discard	`git restore filename`
Committed locally, not pushed yet	`git reset`
Committed AND already pushed	`git revert`
Mid-work, need to switch tasks temporarily	`git stash`

Undoing Uncommitted Changes



```
git restore filename.java
git restore --staged filename.ja
git reset --hard
```

Undoing Local Commits (Not Yet Pushed)

Command	Effect on the commit	Effect on your changes
`git reset --soft HEAD~1`	Removed	Kept, staged (ready to re-commit)
`git reset --mixed HEAD~1`	Removed	Kept, unstaged
`git reset --hard HEAD~1`	Removed	Discarded entirely — gone

`--mixed` is the default if you omit the flag. `git commit --amend` is a related shortcut for fixing the message or contents of only the most recent commit, without a full reset.

Undoing a Commit That's Already Pushed

Never Reset Shared History

Once a commit is pushed and teammates may have pulled it, use `git revert` — never `git reset`. Reset rewrites history; if others have already built on top of the old version, forcing that rewrite onto them causes major, confusing conflicts.

BASH

```
git revert <commit-hash>   # cre
git push
```

Other Everyday Recovery Tools

TEMPORARY SHELVE

git stash

Sets aside uncommitted changes without committing them, so you can switch branches cleanly. `git stash pop`

SELECTIVE COPY

git cherry-pick

Applies one specific commit from another branch onto your current branch, without merging everything

restores them later; `git stash list` shows everything shelved.

else from that branch.

The Ultimate Safety Net: git reflog

`git reflog` shows a log of every place HEAD has pointed to recently — including commits "lost" by a hard reset. As long as Git's garbage collection hasn't run yet, you can almost always recover a commit you thought was gone by finding its hash in the reflog and checking it out.

Accidentally Committed a Secret or Password

Deleting the line in a new commit is not enough — the secret still exists in earlier history and can be found by anyone who inspects the log. Rotate or revoke the exposed credential immediately; treat it as compromised regardless of any cleanup. Removing it from history entirely requires rewriting history (tools like `git filter-repo` or

GitHub's secret-scanning + BFG Repo-Cleaner) — advanced territory, and rotation matters more than the cleanup itself.

CHAPTER 09

GitHub CLI (gh)

Since your terminal environment already has gh authenticated via PAT, most GitHub web actions have a faster terminal equivalent — no browser context-switch required mid-workflow.

Repository Commands

BASH

```
gh auth login
gh repo create my-project --publ
gh repo clone username/repo-name
gh repo view --web
```

Pull Request Commands

BASH

```
gh pr create --title "Add featu
gh pr list
gh pr view 12
```

```
gh pr checkout 12          # pul
gh pr merge 12 --squash
```

Issue Commands

BASH

```
gh issue create
gh issue list
gh issue close 4
```

Why This Fits Your Setup

A terminal-first workflow on a Debian/XFCE4 environment means fewer context switches to a browser for routine actions. `gh pr checkout` in particular is worth memorizing early — testing a teammate's PR locally before approving it is a habit that separates careful reviewers from rubber-stamp ones.

CHAPTER 10

Team Collaboration Playbook

Technical commands are the easy part. What actually causes group-project pain is coordination — and that's solved with a small set of norms, not more Git knowledge.

Branch Protection (Repo Owner Setup)

1. **Go to Settings → Branches**
   On the repo, as an owner/admin.
2. **Add a branch protection rule for `main`**
   Requires PRs before merging — nobody, including the repo owner, can push directly to `main` by accident.
3. **Optionally require review approval**
   At least one teammate must approve before a PR can be merged — enforces the review habit instead of relying on self-discipline.

Adding Collaborators

Repo → Settings → Collaborators → Add people. They must accept an email or GitHub invite before they can push.

Group Project Etiquette

Direct pushes	Never push directly to `main` on a shared repo, even for a "tiny" change — always branch and open a PR, so teammates have visibility into what's changing.
Stale local copy	Pull before you start working each session — starting from an outdated `main` is the single biggest cause of avoidable merge conflicts.
Silent overlap	Two people editing the same file on the same feature without communicating. A quick message ("I'm working on Student.java's validation logic") prevents most conflicts before they happen.
Review as rubber stamp	Approving a PR without actually reading the diff. Even a quick skim catches obvious mistakes and signals the review process is real.

Dividing Work to Minimize Conflicts

Structure feature branches around files or modules different people primarily own, rather than everyone editing the same shared file simultaneously. For a typical BSIT group project (e.g. IPT101/102 integration work), splitting by layer — one person on the database/model layer, another on UI, another on business logic — naturally keeps people out of each other's files most of the time.

Code Review Etiquette

GIVING FEEDBACK

Be Specific, Not Just Critical

"This loop re-queries the database on every iteration — consider fetching once outside the loop" is actionable. "This is

RECEIVING FEEDBACK

Treat It As Improving the Project, Not You

A requested change on a PR is about the code, not a judgment of skill — the entire point of review is catching things a second set of

inefficient" is not.

eyes sees that the author, deep in their own logic, doesn't.

CHAPTER 11

Professional Standards & Security

This is what separates a repo that looks like a school assignment from one that looks like it was built by someone who already works like a professional — relevant well before your first internship application.

README Quality Checklist

1. **What the project does**
   One or two sentences, no assumed context.

2. **How to run it**
   Exact setup commands — someone should be able to clone and run it without asking you a single question.

3 **Tech stack / dependencies**
What language, framework, JDK version, database, etc.

4 **Screenshots (if UI-based)**
Especially valuable for web/UI projects — instant visual context.

Semantic Versioning

Position	Name	Increments when...
MAJOR (1.0.0)	Breaking change	Something incompatible with the previous version was changed
MINOR (0.1.0)	New feature	Functionality added in a backward-compatible way
PATCH (0.0.1)	Bug fix	Backward-compatible bug fix, no new functionality

Common Licenses

License	What it allows
MIT	Very permissive — anyone can use, modify, and distribute, even commercially, with attribution

License	What it allows
Apache 2.0	Similar to MIT, plus explicit patent protection language
GPL v3	Copyleft — derivative works must also be open-sourced under GPL

Security Basics

Non-Negotiables

Enable two-factor authentication (2FA) on your GitHub account. Give Personal Access Tokens the minimum scope they actually need, not full access by default. Rotate any credential the moment it's exposed, even briefly. Never commit `.env` files, API keys, or database passwords — this was covered in Chapter 02's `.gitignore` section for a reason.

Profile Polish

HIGH IMPACT

Profile README

Create a repo with the exact same name as

CURATION

Pinned Repositories

Profile → Customize your pins — choose your 6 best/

your GitHub username (case-sensitive) containing a `README.md` — GitHub automatically displays it at the top of your profile page. The single highest-impact thing for making a profile look intentional rather than empty.	most representative projects, instead of whatever you happened to push most recently.

Commit Hygiene as a Professional Signal

Atomic commits (one logical change each), meaningful Conventional Commits-style messages, and a clean branch history read as competence to anyone reviewing your GitHub activity later — including recruiters who look at commit history, not just the final code.

Complete Command Cheat Sheet & Glossary

Full quick-reference for everything covered in this guide.

Command Reference

Command	Purpose
`git status`	What's changed / staged
`git add .`	Stage everything
`git add -p`	Stage selected chunks interactively
`git commit -m "msg"`	Save a snapshot with a message
`git commit --amend`	Edit the most recent commit
`git push`	Upload commits to GitHub
`git pull`	Download + merge latest changes
`git fetch`	

Command	Purpose
	Download changes without merging
`git log --oneline --graph --all`	Compact, visual commit history
`git diff`	Unstaged line changes
`git diff --staged`	Staged line changes
`git branch`	List branches
`git checkout -b name` / `git switch -c name`	Create + switch to new branch
`git merge branch-name`	Merge a branch into the current one
`git clone url`	Download a repo with full history
`git remote add name url`	Add a remote reference
`git restore file`	Discard uncommitted changes to a file
`git reset --soft/--mixed/--hard HEAD~1`	Undo local commit (varying severity)

Command	Purpose
`git revert <hash>`	Safely undo a pushed commit
`git stash` / `git stash pop`	Temporarily shelve / restore changes
`git cherry-pick <hash>`	Apply one specific commit onto current branch
`git reflog`	Recovery log of everywhere HEAD has pointed
`git tag v1.0.0`	Mark a commit as a named version

Glossary

Repository (repo)	A project's folder, tracked by Git
Commit	A saved snapshot of changes, with a message
Branch	An independent line of development
Merge	Combining changes from one branch into another
Clone	Downloading a full copy of a

Version Control & Collaboration Mastery

Complete Reference · 2026

	remote repo, including history
Fork	Your own copy of someone else's repo, on GitHub
Pull Request (PR)	A proposal to merge changes, with review/ discussion attached
HEAD	Pointer to your current position in history
Origin	Default name for your main remote (usually GitHub)
Upstream	Conventional name for the original repo you forked from
Staging area	Changes marked ready to commit, via `git add`
Conflict	When two branches change the same lines and Git can't auto-merge
Rebase	Replaying commits onto a new base, producing linear history
Squash	

GITHUB · DEV SKILL TRACK

Version Control & Collaboration Mastery

Complete Reference · 2026

INTRODUCTION

CHAPTERS

	Combining multiple commits into one
Reflog	Local log of every place HEAD has pointed — the ultimate undo safety net

Built for **Kian** · Practical Dev Skill Track · Complements WS101/WS102, IPT101/102, SIA101/102, and CAP101/102 group repo work